01.Write a program to search an element from a list. Give user the option to perform Linear or Binary search. Use Template functions.

Answer.

```
#include <stdio.h>
// Function to perform Linear Search
// Works on both sorted and unsorted arrays
int linearSearch(int arr[], int n, int key)
 {
   for (int i = 0; i < n; i++) {
      if (arr[i] == key)
         return i; // return index if element is found
   }
   return -1; // return -1 if element not found
}          // Function to perform Binary Search
int binarySearch(int arr[], int n, int key) // Works only when array is sorted
 {
   int low = 0, high = n - 1, mid;
   while (low <= high)
 {
      mid = (low + high) / 2;
      if (arr[mid] == key)
         return mid; // element found at index mid
      else if (arr[mid] < key)
         low = mid + 1; // search in right half
      else
         high = mid - 1; // search in left half
   }
   return -1; // return -1 if not found
}
int main()
 {
   int n, choice, key, result;        // Input size of array
   printf("Enter number of elements: ");
   scanf("%d", &n);
   int arr[n]; // array declaration
   printf("Enter %d elements (sorted if you want to use Binary Search):\n", n); // Input elements
   for (int i = 0; i < n; i++)
 {
      scanf("%d", &arr[i]);
   }
  printf("Enter element to search: ");// Input element to be searched
   scanf("%d", &key);    // Give user option to choose search method
   printf("Choose search method:\n");
   printf("1. Linear Search\n");
   printf("2. Binary Search\n");
   printf("Enter choice: ");
   scanf("%d", &choice);
  if (choice == 1) // Perform search according to choice
 {
      result = linearSearch(arr, n, key);
   }
else if (choice == 2)
{
      result = binarySearch(arr, n, key);
   }
Else
 {
      printf("Invalid choice!\n");
      return 0;
   }  // Display result
if (result != -1)
      printf("Element found at position %d\n", result + 1); // 1-based index
   else
      printf("Element not found.\n");
  return 0;
}
```

02. WAP using templates to sort a list of elements. Give user the option to perform sorting using Insertion sort, Bubble sort or Selection sort.

Answer.

```
#include <stdio.h>
void bubbleSort(int arr[], int n) // Function to perform Bubble Sort
{
   int i, j, temp;
   for (i = 0; i < n - 1; i++)
{
      for (j = 0; j < n - i - 1; j++)
{
         if (arr[j] > arr[j + 1])
            // Swap arr[j] and arr[j+1]
 {
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }
}
void selectionSort(int arr[], int n) // Function to perform Selection Sort
 {
   int i, j, minIndex, temp;
   for (i = 0; i < n - 1; i++)
 {
      minIndex = i;
      for (j = i + 1; j < n; j++)
{
         if (arr[j] < arr[minIndex])
            minIndex = j;
      }// Swap arr[i] and arr[minIndex]
      temp = arr[i];
      arr[i] = arr[minIndex];
      arr[minIndex] = temp;
   }
}
void insertionSort(int arr[], int n) // Function to perform Insertion Sort
 {
   int i, j, key;
   for (i = 1; i < n; i++)
 {
      key = arr[i];
      j = i - 1;
while (j >= 0 && arr[j] > key) // Shift elements greater than key to one position ahead
{
         arr[j + 1] = arr[j];
         j = j - 1;
      }
      arr[j + 1] = key;
   }
}
void display(int arr[], int n) // Function to display the array
 {
   for (int i = 0; i < n; i++)
      printf("%d ", arr[i]);
   printf("\n");
}
int main()
 {
   int n, choice;
   printf("Enter number of elements: ");    // Input number of elements
   scanf("%d", &n);
   int arr[n]; // Input array elements
   printf("Enter %d elements:\n", n);
   for (int i = 0; i < n; i++)
{
```

```
scanf("%d", &arr[i]);
   }// Menu for sorting methods
printf("Choose Sorting Method:\n");
   printf("1. Bubble Sort\n");
   printf("2. Selection Sort\n");
   printf("3. Insertion Sort\n");
   printf("Enter choice: ");
   scanf("%d", &choice);
   switch (choice) // Perform sorting based on choice
{
      case 1:
         bubbleSort(arr, n);
         printf("Sorted array using Bubble Sort: ");
         break;
      case 2:
         selectionSort(arr, n);
         printf("Sorted array using Selection Sort: ");
         break;
      case 3:
         insertionSort(arr, n);
         printf("Sorted array using Insertion Sort: ");
         break;
      default:
         printf("Invalid choice!\n");
         return 0;
   }// Display sorted array
Display(arr, n);
return 0;
}
```

3. Implement Linked List using templates. Include functions for insertion, deletion and search of a number, reverse the list and concatenate two linked lists (include a function and also overload operator +).

Answer.

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node* next;
};
struct Node* createNode(int val)
 {
   struct Node* n = malloc(sizeof(struct Node));
   n->data = val; n->next = NULL;
   return n;
}
void insert(struct Node** head, int val)
 {
   struct Node* n = createNode(val);
   if (!*head) *head = n;
   else
 {
      struct Node* t = *head;
      while (t->next) t = t->next;
      t->next = n;
   }
}
void del(struct Node** head, int val)
 {
   if (!*head) return;
   struct Node *t = *head, *p = NULL;
   while (t && t->data != val) { p = t; t = t->next;
}
   if (!t) return;
   if (!p) *head = t->next;
   else p->next = t->next;
```

```
   free(t);
}
int search(struct Node* head, int val)
 {
   while (head)
 {
       if (head->data == val) return 1;
       head = head->next;
   }
   return 0;
}
void reverse(struct Node** head)
 {
   struct Node *prev = NULL, *cur = *head, *next = NULL;
   while (cur)
{
       next = cur->next;
       cur->next = prev;
       prev = cur;
       cur = next;
   }
   *head = prev;
}
void concat(struct Node** h1, struct Node* h2)
 {
   if (!*h1) *h1 = h2;
   else
 {
      struct Node* t = *h1;
       while (t->next) t = t->next;
       t->next = h2;
   }
}
void show(struct Node* h)
 {
   while (h) { printf("%d->", h->data); h = h->next;
 }
   printf("NULL\n");
}
int main()
 {
   struct Node *l1 = NULL, *l2 = NULL;
   int ch, val;
   do
{
       printf("\n1.Insert 2.Delete 3.Search 4.Reverse 5.Display 6.Ins2 7.Concatenate 0.Exit\n");
       scanf("%d", &ch);
       switch (ch) {
          case 1: printf("Val: "); scanf("%d", &val); insert(&l1, val); break;
          case 2: printf("Del: "); scanf("%d", &val); del(&l1, val); break;
          case 3: printf("Search: "); scanf("%d", &val);
                printf(search(l1, val) ? "Found\n" : "Not Found\n"); break;
          case 4: reverse(&l1); break;
          case 5: show(l1); break;
          case 6: printf("Val2: "); scanf("%d", &val); insert(&l2, val); break;
          case 7: concat(&l1, l2); printf("After concat: "); show(l1); break;
       }
   }
while (ch != 0);
}
```

4. Implement Doubly Linked List using templates. Include functions for insertion, deletion and search of a number, reverse the list.

Answer.

```
#include <stdio.h>
#include <stdlib.h>
struct Node// Node structure
```

```
 {
   int data;
   struct Node* prev;
   struct Node* next;
};// Insert at end
void insert(struct Node** head, int val)
{
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
   newNode->prev = NULL;
    if (*head == NULL)
{
       *head = newNode;
       return;
   }
   struct Node* temp = *head;
   while (temp->next != NULL)
  {
       temp = temp->next;
   }
   temp->next = newNode;
   newNode->prev = temp;
}// Delete a node
void deleteNode(struct Node** head, int val)
 {
   if (*head == NULL)
{
       printf("List is empty!\n");
       return;
   }
struct Node* temp = *head;    // If head is to be deleted
if (temp->data == val)
 {
       *head = temp->next;
       if (*head != NULL)
          (*head)->prev = NULL;
       free(temp);
       return;
   }
while (temp != NULL && temp->data != val)
       temp = temp->next;
            if (temp == NULL)
{
       printf("Element not found!\n");
       return;
   }
if (temp->next != NULL)
       temp->next->prev = temp->prev;
   if (temp->prev != NULL)
       temp->prev->next = temp->next;
            free(temp);
}// Search element
int search(struct Node* head, int val)
 {
   struct Node* temp = head;
   while (temp != NULL)
 {
       if (temp->data == val)
          return 1;
       temp = temp->next;
   }
   Return 0;
}// Reverse the list

void reverse(struct Node** head)
{
   struct Node* curr = *head;
   struct Node* temp = NULL;
```

```
while (curr != NULL)
 {
      temp = curr->prev; // Swap prev and next
      curr->prev = curr->next;
      curr->next = temp;
      curr = curr->prev;
   }
if (temp != NULL)
      *head = temp->prev;
}// Display list
void display(struct Node* head)
 {
   struct Node* temp = head;
   printf("NULL <-> ");
   while (temp != NULL)
 {
      printf("%d <-> ", temp->data);
      temp = temp->next;
   }
   printf("NULL\n");
}// Main function
int main()
{
   struct Node* list = NULL;
   printf("--- Insertion ---\n");
   insert(&list, 10);
   insert(&list, 20);
   insert(&list, 30);
   display(list);
   printf("--- Deletion of 20 ---\n");
   deleteNode(&list, 20);
   display(list);
   printf("--- Searching 30 ---\n");
   if (search(list, 30))
      printf("Found\n");
   else
      printf("Not Found\n");
   printf("--- Reversing the list ---\n");
   reverse(&list);
   display(list);
   return 0;
}
```

5. Implement Circular Linked List using templates. Include functions for insertion, deletion and search of a number, reverse the list.

Answer.

```
#include <stdio.h>
#include <stdlib.h>

struct Node
 {
   int data;
   struct Node* next;
};// Insert at end
void insert(struct Node** head, int val)
 {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
if (*head == NULL)
 {
      *head = newNode;
      newNode->next = *head;
      return;
   }
   struct Node* temp = *head;
   while (temp->next != *head)
      temp = temp->next;
```

```
        temp->next = newNode;
    newNode->next = *head;
}// Delete node
void deleteNode(struct Node** head, int val)
 {
    if (*head == NULL) return;
    struct Node *curr = *head, *prev = NULL;
    if (curr->data == val) // If head to be deleted
 {
        while (curr->next != *head)
            curr = curr->next;
        struct Node* del = *head;
        curr->next = (*head)->next;
        *head = (*head)->next;
        free(del);
        return;
    }
curr = *head;
    do
{
        prev = curr;
        curr = curr->next;
        if (curr->data == val)
{
            prev->next = curr->next;
            free(curr);
            return;
        }
    } while (curr != *head);
}// Search
int search(struct Node* head, int val)
 {
    if (head == NULL) return 0;
    struct Node* temp = head;
    do
 {
        if (temp->data == val) return 1;
        temp = temp->next;
 }
while (temp != head);
    return 0;
}// Reverse
void reverse(struct Node** head)
{
    if (*head == NULL) return;
    struct Node *prev = NULL, *curr = *head, *next;
    do
{
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
while (curr != *head);
    (*head)->next = prev;
    *head = prev;
}// Display
void display(struct Node* head)
 {
    if (head == NULL) return;
    struct Node* temp = head;
    do
 {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
 while (temp != head);
    printf("(back to head)\n");
}
```

```
int main()
 {
   struct Node* list = NULL;
   insert(&list, 10);
   insert(&list, 20);
   insert(&list, 30);
   display(list);
   deleteNode(&list, 20);
   display(list);
   printf("Search 30: %s\n", search(list, 30) ? "Found" : "Not Found");
    reverse(&list);
   display(list);
    return 0;
}
```

6. Perform Stack operations using Linked List implementation.

Answer.

```
#include <stdio.h>
#include <stdlib.h>
struct Node
 {
   int data;
   struct Node* next;
};
struct Node* top = NULL; // Push
void push(int val)
{
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = top;
   top = newNode;
}// Pop
void pop()
 {
   if (top == NULL)
 {
       printf("Stack Underflow!\n");
       return;
   }
   struct Node* temp = top;
   printf("Popped: %d\n", temp->data);
   top = top->next;
   free(temp);
}// Display
void display()
 {
   struct Node* temp = top;
   printf("Stack: ");
   while (temp != NULL)
 {
       printf("%d ", temp->data);
       temp = temp->next;
   }
   printf("\n");
}
int main()
{
   push(10);
   push(20);
   push(30);
   display();
   pop();
   display();
             return 0;
}
```

7. Perform Stack operations using Array implementation. Use Templates.Answer.

```
#include <stdio.h>
```

```
#define MAX 5
int stack[MAX], top = -1;
void push(int val) // Push
{
   if (top == MAX - 1) {
      printf("Stack Overflow!\n");
      return;
   }
   stack[++top] = val;
}// Pop
void pop()
   if (top == -1)
 {
      printf("Stack Underflow!\n");
      return;
   }
   printf("Popped: %d\n", stack[top--]);
}// Display
void display()
 {
   printf("Stack: ");
   for (int i = top; i >= 0; i--)
      printf("%d ", stack[i]);
   printf("\n");
}
int main()
{
   push(10);
   push(20);
   push(30);
   display();
    pop();
   display();
return 0;
}
```

8. Perform Queues operations using Circular Array implementation. Use Templates.

Answer.

```
#include <stdio.h>
#define MAX 5
int queue[MAX];
int front = -1, rear = -1;
void enqueue(int val) // Enqueue
 {
   if ((rear + 1) % MAX == front)
 {
      printf("Queue Overflow!\n");
      return;
   }
   if (front == -1) front = 0;
   rear = (rear + 1) % MAX;
   queue[rear] = val;
}// Dequeue
void dequeue()
{
   if (front == -1)
 {
      printf("Queue Underflow!\n");
      return;
   }
   printf("Dequeued: %d\n", queue[front]);
   if (front == rear)
      front = rear = -1;
   else
      front = (front + 1) % MAX;
}// Display
void display()
{
```

```
if (front == -1)
 {
      printf("Queue is Empty!\n");
      return;
   }
   printf("Queue: ");
   int i = front;
   while (1) {
      printf("%d ", queue[i]);
      if (i == rear) break;
      i = (i + 1) % MAX;
   }
   printf("\n");
}

int main()
 {
   enqueue(10);
   enqueue(20);
   enqueue(30);
   display();
  dequeue();
   display();
   return 0;
}
```

9. Create and perform different operations on Double-ended Queues using Linked List implementation.

Answer.

```
#include <stdio.h>
#include <stdlib.h>
struct Node
 {
   int data;
   struct Node* prev;
   struct Node* next;
};
struct Node* front = NULL;
struct Node* rear = NULL; // Insert at front
void insertFront(int val)
{
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->prev = NULL;
   newNode->next = front;
   if (front != NULL) front->prev = newNode;
   else rear = newNode;
   front = newNode;
}// Insert at rear
void insertRear(int val)
 {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
   newNode->prev = rear;
   if (rear != NULL) rear->next = newNode;
   else front = newNode;
   rear = newNode;
}// Delete from front
void deleteFront()
 {
   if (front == NULL)
 {
      printf("Deque Underflow!\n");
      return;
   }
   struct Node* temp = front;
```

```
   printf("Deleted: %d\n", temp->data);
   front = front->next;
   if (front != NULL) front->prev = NULL;
   else rear = NULL;
   free(temp);
}// Delete from rear
void deleteRear()
{
   if (rear == NULL)
 {
      printf("Deque Underflow!\n");
      return;
   }
   struct Node* temp = rear;
   printf("Deleted: %d\n", temp->data);
   rear = rear->prev;
   if (rear != NULL) rear->next = NULL;
   else front = NULL;
   free(temp);
}// Display
void display()
 {
   struct Node* temp = front;
   printf("Deque: ");
   while (temp != NULL)
 {
      printf("%d ", temp->data);
      temp = temp->next;
   }
   printf("\n");
}

int main()
{
   insertRear(10);
   insertRear(20);
   insertFront(5);
   display();
  deleteFront();
   display();
   deleteRear();
   display();
   return 0;
}
```

10. WAP to calculate factorial and to compute the factors of a given no. (i) using recursion, (ii) using iteration.

Answer.

```
#include <stdio.h>// Factorial (Recursion)
int factRec(int n) {
   if (n == 0 || n == 1) return 1;
   return n * factRec(n - 1);
}// Factorial (Iteration)
int factIter(int n)
{
   int result = 1;
   for (int i = 1; i <= n; i++)
      result *= i;
   return result;
}// Factors (Recursion)
void factorsRec(int n, int i)
 {
   if (i > n) return;
   if (n % i == 0) printf("%d ", i);
   factorsRec(n, i + 1);
}
```

```
// Factors (Iteration)
void factorsIter(int n)
 {
   for (int i = 1; i <= n; i++)
{
      if (n % i == 0) printf("%d ", i);
   }
}

int main()
 {
   int num = 6;
   printf("Factorial (Recursion): %d\n", factRec(num));
   printf("Factorial (Iteration): %d\n", factIter(num));
   printf("Factors (Recursion): ");
   factorsRec(num, 1);
   printf("\n");
   printf("Factors (Iteration): ");
   factorsIter(num);
   printf("\n");
   return 0;
}
```

11. WAP to display fibonacci series (i) using recursion, (ii) using iteration.

Answer.

```
#include <stdio.h>
int fibRec(int n) // Recursion
 {
   if (n == 0) return 0;
   if (n == 1) return 1;
   return fibRec(n - 1) + fibRec(n - 2);
}
void fibIter(int n) // Iteration
 {
   int a = 0, b = 1, c;
   printf("Fibonacci (Iteration): ");
   for (int i = 0; i < n; i++)
{
      printf("%d ", a);
      c = a + b;
      a = b;
      b = c;
   }
   printf("\n");
}

int main()
 {
   int n = 10;
   printf("Fibonacci (Recursion): ");
   for (int i = 0; i < n; i++)
   printf("%d ", fibRec(i));
   printf("\n");
   fibIter(n);
 return 0;
}
```

12. WAP to calculate GCD of 2 numbers (i) with recursion, (ii) without recursion.

Answer.

```
#include <stdio.h>
void merge(int arr[], int l, int m, int r)
{
   int n1 = m - l + 1, n2 = r - m;
   int L[n1], R[n2];
   for (int i = 0; i < n1; i++) L[i] = arr[l + i];
   for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j]
```

```
  int i = 0, j = 0, k = l;
   while (i < n1 && j < n2)
{
      if (L[i] <= R[j]) arr[k++] = L[i++];
      else arr[k++] = R[j++];
   }
   while (i < n1) arr[k++] = L[i++];
   while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r)
 {
   if (l < r)
{
      int m = (l + r) / 2;
      mergeSort(arr, l, m);
      mergeSort(arr, m + 1, r);
      merge(arr, l, m, r);
   }
}

void display(int arr[], int n)
 {
   for (int i = 0; i < n; i++) printf("%d ", arr[i]);
   printf("\n");
}

int main()
{
   int arr[] = {12, 11, 13, 5, 6, 7};
   int n = sizeof(arr) / sizeof(arr[0]);
   printf("Original: ");
   display(arr, n);

   mergeSort(arr, 0, n - 1);

   printf("Sorted: ");
   display(arr, n);

   return 0;
}
```

13. WAP to create a Binary Search Tree and include following operations in tree:

a. Insertion (Recursive and Iterative Implementation)

b. Deletion by copying

c. Deletion by Merging

d. Search a no. in BST

e. Display its preorder, postorder and inorder traversals Recursively

f. Display its preorder, postorder and inorder traversals Iteratively

g. Display its level-by-level traversals

h. Count the non-leaf nodes and leaf nodes

i. Display height of tree

j. Create a mirror image of tree

k. Check whether two BSTs are equal or not

Answer.

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node
 {
   int data;
   struct Node *left, *right;
};// Create node
struct Node* newNode(int val)
 {
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->data = val;
   node->left = node->right = NULL;
   return node;
}
/*================ Insertion ==================*/// Recursive
struct Node* insertRec(struct Node* root, int val)
{
   if (root == NULL) return newNode(val);
   if (val < root->data) root->left = insertRec(root->left, val);
   else root->right = insertRec(root->right, val);
   return root;
}// Iterative
struct Node* insertItr(struct Node* root, int val)
 {
   struct Node* parent = NULL, *curr = root;
   while (curr != NULL)
{
       parent = curr;
       if (val < curr->data) curr = curr->left;
       else curr = curr->right;
   }
   struct Node* node = newNode(val);
   if (parent == NULL) root = node;
   else if (val < parent->data) parent->left = node;
   else parent->right = node;
   return root;
}/*================ Search ==================*/
struct Node* search(struct Node* root, int key)
{
   if (root == NULL || root->data == key) return root;
   if (key < root->data) return search(root->left, key);
   return search(root->right, key);
}/*================ Traversals ==================*/
void inorder(struct Node* root)
 {
   if (root != NULL)
{
       inorder(root->left);
       printf("%d ", root->data);
       inorder(root->right);
   }
}
void preorder(struct Node* root)
 {
   if (root != NULL)
{
       printf("%d ", root->data);
       preorder(root->left);
       preorder(root->right);
   }
}

void postorder(struct Node* root)
 {
   if (root != NULL)
 {
       postorder(root->left);
       postorder(root->right);
       printf("%d ", root->data);
   }
}
```

```
/*================ Height =================*/
int height(struct Node* root)
{
   if (root == NULL) return 0;
   int lh = height(root->left);
   int rh = height(root->right);
   return (lh > rh ? lh : rh) + 1;
}/*================ Count Leaf & Non-leaf =================*/
void countNodes(struct Node* root, int *leaf, int *nonLeaf)
 {
   if (root == NULL) return;
   if (root->left == NULL && root->right == NULL) (*leaf)++;
   else (*nonLeaf)++;
   countNodes(root->left, leaf, nonLeaf);
   countNodes(root->right, leaf, nonLeaf);
}/*================ Mirror Tree =================*/
struct Node* mirror(struct Node* root)
 {
   if (root == NULL) return NULL;
   struct Node* temp = root->left;
   root->left = mirror(root->right);
   root->right = mirror(temp);
   return root;
}
/*================ Compare Two Trees =================*/
int isEqual(struct Node* t1, struct Node* t2)
 {
   if (t1 == NULL && t2 == NULL) return 1;
   if (t1 != NULL && t2 != NULL)
      return (t1->data == t2->data &&
            isEqual(t1->left, t2->left) &&
            isEqual(t1->right, t2->right));
   return 0;
}
/*================ Deletion by Copying =================*/
struct Node* findMax(struct Node* root)
{
   while (root->right != NULL) root = root->right;
   return root;
}

struct Node* deleteCopy(struct Node* root, int key)
{
   if (root == NULL) return NULL;

   if (key < root->data) root->left = deleteCopy(root->left, key);
   else if (key > root->data) root->right = deleteCopy(root->right, key);
   else {
      if (root->left == NULL)
 {
         struct Node* temp = root->right;
         free(root);
         return temp;
      }
      else if (root->right == NULL)
 {
         struct Node* temp = root->left;
         free(root);
         return temp;
      }
      else
       {
         struct Node* maxLeft = findMax(root->left);
         root->data = maxLeft->data;
         root->left = deleteCopy(root->left, maxLeft->data);
      }
   }
   return root;
}
```

```
/*================ MAIN ==================*/
int main()
{
   struct Node* root = NULL;
// Insertion Recursive
   root = insertRec(root, 50);
   root = insertRec(root, 30);
   root = insertRec(root, 70);
   root = insertRec(root, 20);
   root = insertRec(root, 40);
   root = insertRec(root, 60);
   root = insertRec(root, 80);
   printf("Inorder Traversal: ");
   inorder(root);
   printf("\n");
   printf("Preorder Traversal: ");
   preorder(root);
   printf("\n");
   printf("Postorder Traversal: ");
   postorder(root);
   printf("\n");
// Search
   int key = 40;
   if (search(root, key)) printf("Found %d in BST\n", key);
   else printf("Not Found!\n");
  // Count Nodes
   int leaf = 0, nonLeaf = 0;
   countNodes(root, &leaf, &nonLeaf);
   printf("Leaf Nodes = %d, Non-Leaf Nodes = %d\n", leaf, nonLeaf);
// Height
   printf("Height of tree = %d\n", height(root));
   // Delete Node
   root = deleteCopy(root, 30);
   printf("Inorder after deletion: ");
   inorder(root);
   printf("\n");
   return 0;
}
```

14. WAP to create a Sparse Matrix and represent it using 3-tuple representation

Answer.

```
#include <stdio.h>
#define MAX 100
void sparseMatrix(int rows, int cols, int matrix[rows][cols]) // Function to convert matrix into 3-tuple
 {
   int tuple[MAX][3];
   int i, j, k = 1, count = 0;
   for (i = 0; i < rows; i++)     // Count non-zero elements
{
      for (j = 0; j < cols; j++)
{
         if (matrix[i][j] != 0)
            count++;
      }
   }
   tuple[0][0] = rows;  // First row of tuple: [rows, cols, total non-zero]
   tuple[0][1] = cols;
   tuple[0][2] = count;
   for (i = 0; i < rows; i++) // Fill tuple with non-zero elements
{
      for (j = 0; j < cols; j++)
{
         if (matrix[i][j] != 0)
{
            tuple[k][0] = i;
            tuple[k][1] = j;
            tuple[k][2] = matrix[i][j];
```

```
            k++;
        }
    }
  }

  // Display Tuple Representation
  printf("\n3-Tuple Representation:\n");
  printf("Row Col Value\n");
  for (i = 0; i <= count; i++)
{
    printf("%3d %3d %3d\n", tuple[i][0], tuple[i][1], tuple[i][2]);
  }
}
int main() {
  int matrix[4][5] =
{
    {0, 0, 3, 0, 4},
    {0, 0, 5, 7, 0},
    {0, 0, 0, 0, 0},
    {0, 2, 6, 0, 0}
  };

  printf("Original Matrix:\n");
  for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 5; j++) {
      printf("%3d ", matrix[i][j]);
    }
    printf("\n");
  }
  sparseMatrix(4, 5, matrix);
  return 0;
}
```

15. Implement Skip List using templates. Include functions for insertion, deletion and searching a number.

Answer.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXLVL 3     // Maximum level of skip list
#define PROB 0.5     // Probability for random level
// Node structure for skip list
struct Node
 {
  int key;             // Value stored in node
  struct Node* forward[MAXLVL + 1]; // Array of forward pointers for each level
};
// Create a new node
struct Node* createNode(int key)
{
  struct Node* node = (struct Node*)malloc(sizeof(struct Node));
  node->key = key;
  for (int i = 0; i <= MAXLVL; i++)
    node->forward[i] = NULL; // Initialize forward pointers as NULL
  return node;
}
// Generate random level for node
int randomLevel()
{
  int lvl = 0;
  while (((double)rand() / RAND_MAX) < PROB && lvl < MAXLVL)
    lvl++;
  return lvl;
}// Insert a key into skip list
void insert(struct Node* header, int key)
 {
  struct Node* update[MAXLVL + 1];
```

```
   struct Node* curr = header; // Find place to insert
   for (int i = MAXLVL; i >= 0; i--) {
      while (curr->forward[i] && curr->forward[i]->key < key)
         curr = curr->forward[i];
      update[i] = curr;
   }
   curr = curr->forward[0];
   if (!curr || curr->key != key)
{
      int lvl = randomLevel();        // Random level for new node
      struct Node* newNode = createNode(key);
      for (int i = 0; i <= lvl; i++)
 {
         newNode->forward[i] = update[i]->forward[i];
         update[i]->forward[i] = newNode;
      }
   }
}
// Search for a key in skip list
int search(struct Node* header, int key)
{
   struct Node* curr = header;
   for (int i = MAXLVL; i >= 0; i--)
 {
      while (curr->forward[i] && curr->forward[i]->key < key)
         curr = curr->forward[i];
   }
   curr = curr->forward[0];
   return (curr && curr->key == key); // Return 1 if found, 0 otherwise
}// Display skip list
void display(struct Node* header)
 {
   printf("\nSkip List:\n");
   for (int i = 0; i <= MAXLVL; i++)
{
      struct Node* node = header->forward[i];
      printf("Level %d: ", i);
      while (node)
 {
         printf("%d -> ", node->key);
         node = node->forward[i];
      }
      printf("NULL\n");
   }
}

int main()
 {
   srand(time(0));
   struct Node* header = createNode(-1); // Header node with dummy key
   // Insert elements
   insert(header, 10);
   insert(header, 20);
   insert(header, 30);
   insert(header, 15);
   display(header);  // Display skip list
   int key = 20;     // Search for a key
   if (search(header, key)) printf("\nFound %d\n", key);
   else printf("\nNot Found!\n");
   return 0;
}
```

16. Implement Threaded Binary Tree and perform:

a. Insertion

b. Deletion

c. Inorder Traversal without recursion

Answer.

```
#include <stdio.h>
#include <stdlib.h>
struct Node// Node structure for threaded binary tree
 {
   int data;
   struct Node *left, *right;
   int lthread, rthread;   // Flags: 1 if left/right pointer is a thread
};// Create a new node
struct Node* newNode(int val)
{
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->data = val;
   node->left = node->right = NULL;
   node->lthread = node->rthread = 1; // Initially threads
   return node;
}
// Insert a new node into threaded binary tree
struct Node* insert(struct Node* root, int key)
{
   struct Node* parent = NULL, *curr = root;
   while (curr != NULL)     // Find the position to insert
{
      if (key == curr->data) return root; // Duplicate keys not allowed
      parent = curr;
      if (key < curr->data)
 {
         if (curr->lthread == 0)
            curr = curr->left;
         else break;
      }
else
{
         if (curr->rthread == 0)
            curr = curr->right;
         else break;
      }
   }
   struct Node* node = newNode(key);     // If tree is empty
   if (parent == NULL) root = node;
   else if (key < parent->data)
 {
      node->left = parent->left;   // Left thread
      node->right = parent;       // Right points to parent
      parent->lthread = 0;        // Left is no longer a thread
      parent->left = node;
   }
else
{
      node->right = parent->right; // Right thread
      node->left = parent;        // Left points to parent
      parent->rthread = 0;        // Right is no longer a thread
      parent->right = node;
   }
   return root;
}
// Find inorder successor
struct Node* inorderSuccessor(struct Node* ptr)
 {
   if (ptr->rthread == 1) return ptr->right; // If right is thread
   ptr = ptr->right;
   while (ptr->lthread == 0) // Go to leftmost node in right subtree
      ptr = ptr->left;
   return ptr;
}
// Inorder traversal without recursion
void inorder(struct Node* root)
{
   if (root == NULL) return;
```

```
   struct Node* curr = root;
   while (curr->lthread == 0) // Go to leftmost node
      curr = curr->left;
   while (curr != NULL)
{
      printf("%d ", curr->data);
      curr = inorderSuccessor(curr);
   }
}// Delete a node from TBT
struct Node* deleteNode(struct Node* root, int key)
{
   struct Node *parent = NULL, *curr = root;
// Search for node to delete
   while (curr != NULL && curr->data != key)
{
      parent = curr;
      if (key < curr->data)
 {
         if (curr->lthread == 0)
            curr = curr->left;
         else break;
      }
Else
 {
         if (curr->rthread == 0)
            curr = curr->right;
         else break;
      }
   }

   if (curr == NULL || curr->data != key)
{
      printf("\nKey %d not found.\n", key);
      return root;
   }// Case 1: Node with two children
   if (curr->lthread == 0 && curr->rthread == 0)
 {
      struct Node* succ = inorderSuccessor(curr);
      int val = succ->data;
      root = deleteNode(root, succ->data); // Recursively delete successor
      curr->data = val;
      return root;
   }
// Case 2 & 3: Node with 0 or 1 child
   struct Node* child = NULL;
   if (curr->lthread == 0)
      child = curr->left;
   else if (curr->rthread == 0)
      child = curr->right;
   if (parent == NULL) // Deleting root
{
      free(curr);
      return child;
   }

   if (curr == parent->left)
 {
      if (child != NULL)
         parent->left = child;
      else
 {
         parent->lthread = 1;
         parent->left = curr->left;
      }
   }
else
{
      if (child != NULL)
```

```
        parent->right = child;
      else
{
        parent->rthread = 1;
        parent->right = curr->right;
      }
   }

   free(curr);
   return root;
}
int main()
 {
   struct Node* root = NULL;
   // Insert nodes
   root = insert(root, 20);
   root = insert(root, 10);
   root = insert(root, 30);
   root = insert(root, 5);
   root = insert(root, 15);
   printf("Inorder Traversal before deletion: ");
   inorder(root);
   printf("\n");
   // Delete a node
   root = deleteNode(root, 10);
   printf("Inorder Traversal after deletion: ");
   inorder(root);
   printf("\n");
   return 0;
}
```

17. Implement AVL Tree and perform:

a. Insertion with balancing (LL, RR, LR, RL rotations)

b. Deletion with balancing

c. Traversals

Answer.

```
#include <stdio.h>
#include <stdlib.h>
struct Node // Node structure for AVL Tree
{
   int data;
   struct Node *left;
   struct Node *right;
   int height;
};
// Utility function to get maximum
int max(int a, int b)
{
   return (a > b) ? a : b;
}// Get height of node
int height(struct Node* N)
{
   if (N == NULL) return 0;
   return N->height;
}// Create new node
struct Node* newNode(int key)
 {
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
   node->data = key;
   node->left = node->right = NULL;
   node->height = 1; // New node is initially at leaf
   return node;
}
```

```
// Right rotate
struct Node* rightRotate(struct Node* y)
{
   struct Node* x = y->left;
   struct Node* T2 = x->right;
   x->right = y;
   y->left = T2;
   y->height = max(height(y->left), height(y->right)) + 1;
   x->height = max(height(x->left), height(x->right)) + 1;
   return x;
}// Left rotate
struct Node* leftRotate(struct Node* x)
{
   struct Node* y = x->right;
   struct Node* T2 = y->left;
   y->left = x;
   x->right = T2;
   x->height = max(height(x->left), height(x->right)) + 1;
   y->height = max(height(y->left), height(y->right)) + 1;
   return y;
}
// Get balance factor
int getBalance(struct Node* N)
 {
   if (N == NULL) return 0;
   return height(N->left) - height(N->right);
}// Insert node
struct Node* insert(struct Node* node, int key)
{
   if (node == NULL) return newNode(key);
   if (key < node->data) node->left = insert(node->left, key);
   else if (key > node->data) node->right = insert(node->right, key);
   else return node; // Duplicate keys not allowed
   node->height = 1 + max(height(node->left), height(node->right));
   int balance = getBalance(node);
   // LL Case
   if (balance > 1 && key < node->left->data) return rightRotate(node);
   // RR Case
   if (balance < -1 && key > node->right->data) return leftRotate(node);
   // LR Case
   if (balance > 1 && key > node->left->data)
{
      node->left = leftRotate(node->left);
      return rightRotate(node);
   }
   // RL Case
   if (balance < -1 && key < node->right->data)
{
      node->right = rightRotate(node->right);
      return leftRotate(node);
   }
   return node;
}
// Find minimum value node
struct Node* minValueNode(struct Node* node)
{
   struct Node* current = node;
   while (current->left != NULL)
      current = current->left;
   return current;
}// Delete node
struct Node* deleteNode(struct Node* root, int key)
{
   if (root == NULL) return root;
   if (key < root->data)
      root->left = deleteNode(root->left, key);
   else if (key > root->data)
      root->right = deleteNode(root->right, key);
   else
```

```
{
      // Node with one child or no child
      if (root->left == NULL || root->right == NULL)
 {
         struct Node* temp = root->left ? root->left : root->right;
         if (temp == NULL) // No child
 {
            temp = root;
            root = NULL;
         }
else // One child
            *root = *temp;
         free(temp);
      }
else
{
         struct Node* temp = minValueNode(root->right); // Node with two children
         root->data = temp->data;
         root->right = deleteNode(root->right, temp->data);
      }
   }
   if (root == NULL) return root;    // Update height
   root->height = 1 + max(height(root->left), height(root->right));
   int balance = getBalance(root);
   // Balance the tree
   // LL Case
   if (balance > 1 && getBalance(root->left) >= 0) return rightRotate(root);
   // LR Case
   if (balance > 1 && getBalance(root->left) < 0) {
      root->left = leftRotate(root->left);
      return rightRotate(root);
   }
   // RR Case
   if (balance < -1 && getBalance(root->right) <= 0) return leftRotate(root);
   // RL Case
   if (balance < -1 && getBalance(root->right) > 0) {
      root->right = rightRotate(root->right);
      return leftRotate(root);
   }
   return root;
}
// Inorder traversal
void inorder(struct Node* root)
{
   if (root != NULL)
 {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}
// Preorder traversal
void preorder(struct Node* root)
 {
   if (root != NULL)
 {
      printf("%d ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}
// Postorder traversal
void postorder(struct Node* root)
 {
   if (root != NULL)
{
      postorder(root->left);
      postorder(root->right);
      printf("%d ", root->data);
```

```
  }
}
// Main function
int main()
{
   struct Node* root = NULL;
   // Insertion
   root = insert(root, 10);
   root = insert(root, 20);
   root = insert(root, 30);
   root = insert(root, 40);
   root = insert(root, 50);
   root = insert(root, 25);
   printf("Inorder traversal: ");
   inorder(root);
   printf("\n");
   printf("Preorder traversal: ");
   preorder(root);
   printf("\n");
   printf("Postorder traversal: ");
   postorder(root);
   printf("\n");
   // Deletion
   root = deleteNode(root, 40);
   printf("Inorder after deleting 40: ");
   inorder(root);
   printf("\n");
   return 0;
}
```

18. Implement Hash Table using different techniques:

a. Open Addressing (Linear, Quadratic, Double Hashing)

b. Separate Chaining

Answer.

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 10 // Size of hash table
struct Node// Node structure for Separate Chaining
 {
   int key;
   struct Node* next;
};
int hash(int key) // Primary hash function
 {
   return key % SIZE; // Simple modulo-based hash
}
/* ================= Linear Probing ================== */
void linearProbing()
{
   int table[SIZE]; // Initialize table with -1 (indicates empty slot)
   for (int i = 0; i < SIZE; i++) table[i] = -1;
   int keys[] = {10, 20, 30, 25, 35};
   int n = 5;
   // Insert keys
   for (int i = 0; i < n; i++)
 {
      int index = hash(keys[i]); // If collision, move to next slot linearly
      while (table[index] != -1)
         index = (index + 1) % SIZE;
      table[index] = keys[i];
   }
   // Display table
   printf("Hash Table (Linear Probing):\n");
   for (int i = 0; i < SIZE; i++)
      printf("%d: %d\n", i, table[i]);
```

```
}
/* ================= Quadratic Probing ================== */
void quadraticProbing()
{
   int table[SIZE];
   for (int i = 0; i < SIZE; i++) table[i] = -1;
   int keys[] = {10, 20, 30, 25, 35};
   int n = 5;
   for (int i = 0; i < n; i++)
{
      int index = hash(keys[i]);
      int j = 0;
      int newIndex = index;
      // Resolve collision using quadratic formula: (index + j^2) % SIZE
      while (table[newIndex] != -1)
{
         j++;
         newIndex = (index + j * j) % SIZE;
      }
      table[newIndex] = keys[i];
   }

   printf("\nHash Table (Quadratic Probing):\n");
   for (int i = 0; i < SIZE; i++)
      printf("%d: %d\n", i, table[i]);
}
/* ================= Double Hashing ================== */
int hash2(int key) // Secondary hash function
{
   return 7 - (key % 7); // Step size for double hashing
}
void doubleHashing()
 {
   int table[SIZE];
   for (int i = 0; i < SIZE; i++) table[i] = -1;
   int keys[] = {10, 20, 30, 25, 35};
   int n = 5;
   for (int i = 0; i < n; i++)
 {
      int index = hash(keys[i]);
      int step = hash2(keys[i]);
      int j = 0;
      int newIndex = index;
      // Resolve collision using double hashing formula: (index + j * step) % SIZE
      while (table[newIndex] != -1)
{
         j++;
         newIndex = (index + j * step) % SIZE;
      }
      table[newIndex] = keys[i];
   }

   printf("\nHash Table (Double Hashing):\n");
   for (int i = 0; i < SIZE; i++)
      printf("%d: %d\n", i, table[i]);
}
/* ================= Separate Chaining ================== */
void separateChaining() {
   struct Node* table[SIZE];
   // Initialize all buckets as NULL
   for (int i = 0; i < SIZE; i++) table[i] = NULL;
   int keys[] = {10, 20, 30, 25, 35};
   int n = 5;
   for (int i = 0; i < n; i++) {
      int index = hash(keys[i]);
      // Create new node for chaining
      struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
      newNode->key = keys[i];
      // Insert at beginning of linked list
```

```
        newNode->next = table[index];
        table[index] = newNode;
    }
    printf("\nHash Table (Separate Chaining):\n");// Print each bucket
    for (int i = 0; i < SIZE; i++)
 {
        printf("%d: ", i);
        struct Node* curr = table[i];
        while (curr)
 {
            printf("%d -> ", curr->key);
            curr = curr->next;
        }
        printf("NULL\n");
    }
}

int main(
) {
    // Linear Probing example
    linearProbing();
    // Quadratic Probing example
    quadraticProbing();
    // Double Hashing example
    doubleHashing();
    // Separate Chaining example
    separateChaining();
    return 0;
}
```

19.WAP using Stacks to perform Expression operations:

a. Convert Infix to Postfix

b. Convert Infix to Prefix

c. Evaluate Postfix Expression

d. Evaluate Prefix Expression

Answer.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX 100
// ------------------ Stack for operators ------------------
char stack[MAX];
int top = -1;
void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }
char peek() { return stack[top]; }
int isEmpty() { return top == -1; }
// Operator precedence
int precedence(char c)
{
    if(c=='+'||c=='-') return 1;
    if(c=='*'||c=='/') return 2;
    return 0;
}
// ------------------ Infix to Postfix ------------------
void infixToPostfix(char* infix, char* postfix)
 {
    int i=0,j=0;
    char c;
    top=-1; // reset stack
    while(infix[i])
 {
```

```
        c=infix[i];
        if(isalnum(c)) postfix[j++]=c; // Operand
        else if(c=='(') push(c);
        else if(c==')')
{
            while(!isEmpty() && peek()!='(') postfix[j++]=pop();
            pop(); // remove '('
        }
        Else// operator
 {
            while(!isEmpty() && precedence(peek())>=precedence(c))
                postfix[j++]=pop();
            push(c);
        }
        i++;
    }
    while(!isEmpty()) postfix[j++]=pop();
    postfix[j]='\0';
}

// ------------------ Infix to Prefix ------------------
void reverseString(char* str)
{
    int n = strlen(str);
    for(int i=0;i<n/2;i++)
{
        char temp = str[i];
        str[i] = str[n-1-i];
        str[n-1-i] = temp;
    }
}
void infixToPrefix(char* infix, char* prefix)
{
    char revInfix[MAX], revPostfix[MAX];
    strcpy(revInfix, infix);
    reverseString(revInfix);
    for(int i=0;i<strlen(revInfix);i++)// Swap '(' with ')
'{
        if(revInfix[i]=='(') revInfix[i]=')';
        else if(revInfix[i]==')') revInfix[i]='(';
    }
    infixToPostfix(revInfix, revPostfix);
    reverseString(revPostfix);
    strcpy(prefix, revPostfix);
}
// ------------------ Evaluate Postfix ------------------
int evalPostfix(char* postfix)
 {
    int stackEval[MAX], topEval=-1;
    int i=0;
    char c;
    while(postfix[i])
 {
        c=postfix[i];
        if(isdigit(c)) stackEval[++topEval]=c-'0';
        else
 {
            int b=stackEval[topEval--];
            int a=stackEval[topEval--];
            if(c=='+') stackEval[++topEval]=a+b;
            else if(c=='-') stackEval[++topEval]=a-b;
            else if(c=='*') stackEval[++topEval]=a*b;
            else if(c=='/') stackEval[++topEval]=a/b;
        }
        i++;
    }
    return stackEval[topEval];
}
```

```
// ------------------ Evaluate Prefix ------------------
int evalPrefix(char* prefix)
 {
   int stackEval[MAX], topEval=-1;
   int n = strlen(prefix);
   for(int i=n-1;i>=0;i--)
{
      char c = prefix[i];
      if(isdigit(c)) stackEval[++topEval]=c-'0';
      else
{
         int a = stackEval[topEval--];
         int b = stackEval[topEval--];
         if(c=='+') stackEval[++topEval]=a+b;
         else if(c=='-') stackEval[++topEval]=a-b;
         else if(c=='*') stackEval[++topEval]=a*b;
         else if(c=='/') stackEval[++topEval]=a/b;
      }
   }
   return stackEval[topEval];
}
// ------------------ Main Function ------------------
int main()
 {
   char infix[MAX], postfix[MAX], prefix[MAX];
   printf("Enter Infix Expression (single-digit operands): ");
   scanf("%s", infix);
   // Infix to Postfix
   infixToPostfix(infix, postfix);
   printf("Postfix: %s\n", postfix);
   printf("Evaluated Result (Postfix): %d\n", evalPostfix(postfix));
   // Infix to Prefix
   infixToPrefix(infix, prefix);
   printf("Prefix: %s\n", prefix);
   printf("Evaluated Result (Prefix): %d\n", evalPrefix(prefix));
   return 0;
}
```

20.Implement Shell Sort

Answer.

```
#include <stdio.h>
void shellSort(int arr[], int n) // Function to perform Shell Sort
 {
   for (int gap = n/2; gap > 0; gap /= 2) // Start with a big gap, then reduce the gap
 {
      for (int i = gap; i < n; i++)      // Perform insertion sort for this gap size
 {
         int temp = arr[i];
         int j;          // Shift earlier gap-sorted elements up until the correct location for arr[i] is found
         for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)
            arr[j] = arr[j - gap];
         arr[j] = temp;
      }
   }
}
void display(int arr[], int n) // Function to display array
 {
   for (int i = 0; i < n; i++)
      printf("%d ", arr[i]);
   printf("\n");
}
int main()
{
   int arr[] = {12, 34, 54, 2, 3};
   int n = sizeof(arr)/sizeof(arr[0]);
   printf("Original Array: ");
   display(arr, n);
```

```
    // Sorting the array using Shell Sort
    shellSort(arr, n);
    printf("Sorted Array: ");
    display(arr, n);
    return 0;
}
```